INTELLIGENT SOFTWARE DEFINED NETWORK TRAFFIC MANAGEMENT FOR REDUCING NETWORK CONGESTION AND IMPROVING BANDWIDTH UTILIZATION

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

Course Code & Course Name

to the award of the degree of

BACHELOR OF TECHNOLOGY IN

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

Sree B (192524023)

Under the Supervision of

Guide Name

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Intelligent Software Defined Network For Reducing Network Congestion and Improving Bandwidth Utilization" has been carried out by Sree B (192524023) under the supervision of Guide Name and is submitted in partial fulfilment of the requirements for the current semester of the B-Tech Electronics and Communication Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. T.J. Naga Lakshmi,
Program Director,
Department of ECE,
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. R. Senthamil Selvan,
Associative Professor,
Department of ECE,
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, **Sree(192524023), Srimathi (192511448), Thadi Venkata Sai Kalpana (192425392), Tulpakala Nikhil Kumar (192412139) and Thamisetty Karimulla (192465046)** of the **Computer Science and Engineering Department**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled **"Intelligent Software Defined Network Traffic Management For Reducing Network Congestion And Improving Bandwidth Utilization"** is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Sree B (192524023)

Signature of the Students with Names

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(

Student Name / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Contribution (%)
Sree (192524023)	Implementation	Design	Testing	Documentation	20%
Srimathi (192511448)	Monitoring	Development	Analysis	Literature Review	20%
Thadi Venkata Sai Kalpana (192425392)	Optimization	Development	Evaluation	Results	20%
Tulpakala Nikhil Kumar (192412139)	Integration	Integration	Validation	System Design	20%
Thamisetty Karimulla (192465046)	Security	Configuration	Performance Testing	Conclusion	20%
Total	—	—	—	—	100%

ABSTRACT

Software Defined Networking (SDN) provides a flexible approach to managing modern computer networks by separating the control plane from the data plane. However, traditional networks often experience problems such as network congestion, inefficient bandwidth utilization, limited centralized control, and difficulty in dynamically managing traffic. These challenges increase network delays and reduce overall network performance, creating a need for an intelligent traffic management solution.

This project, **"Intelligent Software Defined Network Traffic Management for Reducing Network Congestion and Improving Bandwidth Utilization,"** proposes an SDN-based solution for efficient network traffic control. The system uses a centralized SDN controller based on **OS-Ken**, **Open vSwitch (OVS)**, and the **OpenFlow 1.3 protocol**. A virtual network environment is developed using **Ubuntu and Mininet**, where hosts communicate through an Open vSwitch controlled by the SDN controller.

The methodology involves designing the SDN architecture, configuring the virtual network, establishing communication between the controller and the Open vSwitch, monitoring network traffic, and managing packet forwarding through programmable flow rules. The centralized controller enables intelligent decision-making and dynamic traffic management based on network conditions. This approach provides better visibility and control compared with conventional distributed network architectures.

The implemented prototype demonstrates successful communication between the SDN controller, Open vSwitch, and virtual hosts. The experimental environment validates the basic functionality of centralized traffic control and programmable flow management. The system provides a foundation for reducing congestion and improving the efficient utilization of available network resources.

In conclusion, the project demonstrates that Software Defined Networking can provide a flexible and programmable solution for modern traffic management problems. The proposed system successfully integrates virtualization, centralized control, and OpenFlow-based communication. Future improvements can incorporate machine learning algorithms, real-time congestion prediction, and larger network topologies to create a more advanced and autonomous traffic management system.

Keywords

Software Defined Networking (SDN); Intelligent Traffic Management; Network Congestion; Bandwidth Utilization; OpenFlow; OS-Ken

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

- The project titled **"Intelligent Software Defined Network Traffic Management for Reducing Network Congestion and Improving Bandwidth Utilization"** focuses on developing an intelligent traffic management system using SDN principles. The proposed system monitors network traffic and identifies congestion or excessive bandwidth utilization. Based on the current network conditions, the SDN controller can make appropriate routing and traffic-management decisions to distribute traffic more efficiently across the available network resources.
- The implementation uses technologies such as **Open vSwitch (OVS)**, an **SDN controller based on OS-Ken**, and a virtualized networking environment. Open vSwitch acts as a programmable network switch, while the controller manages traffic flow and communicates with the switch using the OpenFlow protocol. By dynamically controlling network traffic, the system aims to reduce congestion, improve bandwidth utilization, minimize packet loss, and enhance overall network performance. This project demonstrates how intelligent SDN-based traffic management can provide a more flexible and efficient alternative to traditional network management approaches.

1.2 Need for the Project

The increasing network traffic caused by cloud computing, video streaming, and IoT applications can lead to congestion, delays, packet loss, and poor bandwidth utilization. This problem affects network administrators, organizations, businesses, and end users.

Traditional networks have limited flexibility and often require manual configuration to manage changing traffic conditions. The proposed SDN-based system provides centralized and intelligent traffic management to dynamically control network flows. This project is engineering-significant because it helps reduce congestion, improve bandwidth utilization, and enhance overall network performance.

1.3 Problem Statement

- Traditional networks struggle to manage rapidly changing traffic, causing congestion, packet loss, increased delay, and inefficient bandwidth utilization. The engineering challenge is to develop an **intelligent SDN-based traffic management system** that monitors network conditions and dynamically controls traffic flows.
- The system must balance conflicting requirements such as reducing congestion while maintaining low latency and efficiently utilizing limited network resources. It must also operate under realistic constraints, including limited bandwidth, network topology, controller processing capability, and compatibility between OpenFlow switches and the SDN controller.

1.4 Project Aim

- To develop an intelligent Software Defined Network (SDN) traffic management system.
- To reduce network congestion by monitoring and controlling network traffic dynamically.
- To improve the efficient utilization of available bandwidth.
- To reduce packet loss and network delays.
- To use an SDN controller and OpenFlow-based switches for centralized traffic management.
- To improve the overall performance and reliability of the network.

1.5 Project Objectives

1. To design an SDN-based network architecture for intelligent traffic management.
2. To develop a centralized traffic monitoring and control mechanism using an SDN controller.
3. To model network traffic conditions and identify congestion based on network performance parameters.
4. To implement Open vSwitch and OpenFlow-based communication for dynamic traffic management.
5. To test the implemented system using virtual network hosts and simulated network traffic.

1.6 Scope

- **What is Included**
- Design and implementation of an SDN-based traffic management system.
- Use of Open vSwitch, OpenFlow, and an SDN controller.
- Virtual network topology with multiple hosts.
- Monitoring of network traffic and congestion.
- Evaluation of bandwidth utilization, delay, packet loss, and throughput.

What is Excluded

- Deployment on a large real-world physical network.
- Advanced hardware-based network infrastructure.
- Commercial-level security and intrusion detection features.

Assumptions

- Network devices support OpenFlow communication.
- The SDN controller is available and functioning correctly.
- The virtual network environment represents basic real-world traffic conditions.

Boundary Conditions

- The implementation is limited to a virtualized network environment.
- Available CPU, memory, and bandwidth are limited by the host system.
- Testing is performed using a limited number of virtual switches and hosts.

1.7 Expected Engineering Outcome

System

An intelligent **Software Defined Network (SDN) Traffic Management System** that monitors and controls network traffic centrally.

Product

A software-based solution for reducing network congestion and improving bandwidth utilization.

Process

The system monitors network traffic, identifies congestion, and dynamically manages traffic flows through the SDN controller.

Prototype

A virtual SDN prototype using **Open vSwitch (OVS)**, **OpenFlow 1.3**, an SDN controller, and virtual network hosts.

Algorithm

A traffic-management algorithm that analyzes network conditions and selects appropriate forwarding paths to reduce congestion.

Model

A centralized SDN architecture consisting of an SDN controller, OpenFlow switch, and connected network hosts.

Infrastructure Solution

A virtual network infrastructure implemented on a Windows system using an Ubuntu virtual machine, Open vSwitch, and virtual networking components.

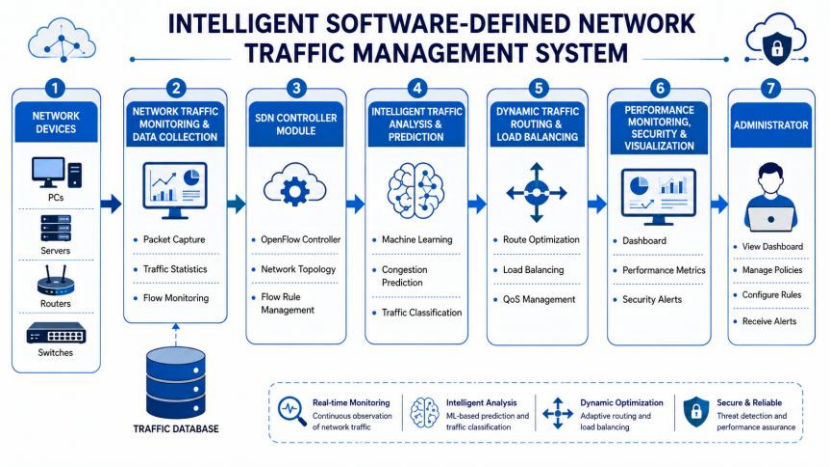

Figure 1: System Architecture

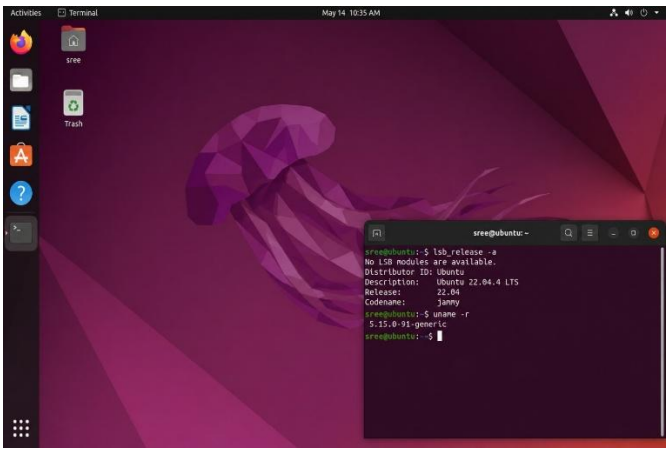

Figure 2: Ubuntu Servr

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature and existing technologies were identified through research papers, technical documentation, networking standards, and studies related to Software Defined Networking (SDN), OpenFlow, network congestion, and bandwidth utilization. Existing technologies such as Open vSwitch and SDN controllers were also reviewed to understand their advantages and limitations.

2.2 Review of Existing Work

Research Papers

Several research studies have explored the use of SDN for network traffic management and congestion control. These studies demonstrate that centralized control enables dynamic routing and better resource utilization. However, some proposed approaches are complex and require advanced infrastructure.

Existing Systems

Traditional network management systems use distributed routing protocols and independently operating network devices. Although these systems are reliable, they have limited centralized control and may not respond efficiently to rapidly changing traffic conditions.

Commercial Solutions

Commercial networking solutions provide advanced traffic monitoring and management features. However, they can require expensive hardware, licenses, and specialized infrastructure, making them unsuitable for small-scale or academic implementations.

Technologies

Technologies such as **OpenFlow**, **Open vSwitch (OVS)**, and SDN controllers enable programmable network management. OpenFlow allows communication between the controller and network switches, while OVS provides a virtual programmable switching environment.

Methods

Common traffic management methods include static routing, load balancing, dynamic routing, and congestion-aware routing. Static approaches are simple but cannot effectively adapt to changing network conditions, whereas dynamic approaches provide better performance but increase system complexity.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Traditional Routing	Simple and reliable	Limited centralized control	Shows limitations of conventional networks
Static Traffic Management	Easy to implement	Cannot adapt to traffic changes	Used for comparison
Load Balancing	Distributes network traffic	May require complex configuration	Supports efficient bandwidth utilization
Dynamic Routing	Adapts to network conditions	Higher computational complexity	Relevant for intelligent traffic control
Commercial SDN Solutions	Advanced features and scalability	Expensive infrastructure	Provides industrial reference
OpenFlow-Based SDN	Centralized and programmable	Requires compatible infrastructure	Core technology of the project
Open vSwitch	Flexible and open-source	Mainly used in virtualized environments	Used for project implementation

Table 1: Comparative Analysis

2.4 Research / Engineering Gap

Existing traffic management approaches often lack a simple and cost-effective solution for dynamically managing congestion in small-scale networks. Traditional networks have limited centralized control, while commercial SDN solutions can be expensive and complex.

There is therefore a need for an affordable and practical SDN-based traffic management system that can monitor network conditions and dynamically manage traffic to reduce congestion and improve bandwidth utilization.

.

2.5 Proposed Contribution

The proposed project develops an intelligent and cost-effective SDN-based traffic management prototype using **Open vSwitch, OpenFlow, and an SDN controller**.

Unlike traditional static network management approaches, the proposed system provides centralized control and enables dynamic traffic management. The system aims to monitor network conditions, identify congestion, and manage traffic flows efficiently.

The main contribution of the project is the implementation of a practical SDN prototype that demonstrates how centralized and programmable network control can help reduce congestion and improve bandwidth utilization in a virtual network environment.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Network Administrator	Monitor and manage network traffic efficiently
End Users	Reduced network delay and improved connectivity
Organization	Efficient utilization of available bandwidth
Project Developer	Easy-to-implement and low-cost SDN solution
System Administrator	Centralized control and monitoring of network devices

Table 1: Stakeholders and User Requirements

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Monitor network traffic	Functional	Detect traffic flow in the network
Identify network congestion	Functional	Detect high traffic conditions
Manage traffic dynamically	Functional	Update forwarding rules dynamically
Control Open vSwitch	Functional	Support OpenFlow 1.3
Reduce packet loss	Functional	Lower packet loss during testing
Improve bandwidth utilization	Functional	More efficient use of available bandwidth
System response	Non-Functional	Quick response to traffic changes
Reliability	Non-Functional	Stable operation during testing
Scalability	Non-Functional	Support multiple virtual hosts
Usability	Non-Functional	Easy configuration and monitoring

Table 2: Functional and Non Functional Requirements

3.2 Constraints & Applicable Standards

Constraints

The project is implemented using a virtualized environment on a Windows system. Therefore, performance is limited by the available CPU, RAM, and virtual network resources. The system is also dependent on compatibility between Ubuntu, Open vSwitch, the SDN controller, and OpenFlow.

Standard / Code	Requirement	How Applied in This Project
OpenFlow 1.3	Standard communication between controller and switch	Used for communication with Open vSwitch
IEEE 802.3	Ethernet communication standard	Applied to Ethernet-based virtual network communication
TCP/IP Protocol Suite	Standard network communication	Used for IP communication between virtual hosts
IPv4	Internet addressing	Virtual hosts use Ipv4 addresses
Linux Networking Standards	Virtual interface management	Used for configuring virtual network interfaces

Table 4: Constraints and Applicable Standards

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
SDN Protocol	OpenFlow 1.3	Version	High
Virtual Switch	Open vSwitch	Software	High
Number of Switches	Minimum 1	Switch	High
Number of Hosts	Minimum 2	Hosts	High
Controller	OS-Ken-based SDN Controller	Software	High
Network Addressing	IPv4	Protocol	High
Congestion Monitoring	Traffic-based detection	Logical	High
Packet Loss	Minimized during testing	Percentage	Medium
Network Delay	**Reduced compared with congested conditions**	**Milliseconds**	**Medium**

Table 5: Design Specification

3.4 Alternative Solutions & Evaluation

Alternative 1: Traditional Network Management

Traditional routers and switches independently manage traffic using distributed routing protocols. This approach is widely used but provides limited centralized control.

Alternative 2: SDN-Based Traffic Management

An SDN controller centrally monitors and manages network traffic through programmable switches. This approach provides flexibility and dynamic traffic control.

Alternative 3: Commercial Network Management Solution

Commercial solutions provide advanced traffic analysis and management features but often require expensive hardware, licenses, and specialized infrastructure.

Evaluation Matrix

Scores: **1 = Poor, 5 = Excellent**

Criterion	**Weight**	**Alternative 1 Traditional Network**	**Alternative 2 SDN-Based System**	**Alternative 3 Commercial Solution**
Performance	0.30	3	5	5
Cost	0.25	4	5	2
Safety	0.10	5	5	5
Sustainability	0.10	3	5	3
Reliability	0.25	4	4	5
Weighted Score	**1.00**	**3.70**	**4.75**	**4.00**

Table 6: Evaluation Matrix

The SDN-based approach achieves the highest weighted score and is therefore selected for implementation.

3.5 Selected Design & Detailed Design

- The SDN controller acts as the centralized decision-making component. Open vSwitch functions as the programmable data-plane switch, while the virtual hosts generate and receive network traffic. OpenFlow 1.3 enables communication between the controller and the switch.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Use SDN	Traditional networking	Centralized and programmable control	Requires controller setup	SDN selected for dynamic traffic management
Open vSwitch	Physical switch	Low cost and easy virtualization	Limited real-world hardware testing	OVS selected for prototype implementation
Virtual environment	Physical infrastructure	Low cost and safe testing	Limited scalability	Virtual environment selected
OpenFlow 1.3	Static routing	Dynamic flow management	Configuration complexity	OpenFlow selected for controller-switch communication
Centralized controller	Distributed control	Global network visibility	Possible single point of failure	Selected for prototype simplicity

Table 7: Trade-Off Analysis

3.7 Sustainability, Ethics, Safety, Risk & Security

This is the section institutional guidelines flag most for vague statements. Never write 'sustainability was considered' — the table must show what evidence was evaluated and how it changed the design decision.

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Energy and hardware consumption	Virtualized implementation reduces the need for additional physical devices	Virtual environment selected
Ethics	Responsible use of network monitoring	Only test network traffic is monitored	No personal user data is collected
Health & Safety	Physical and electrical safety	Software-based implementation has minimal physical risk	Virtual implementation preferred
Security	Unauthorized network access	OpenFlow controller and switch communication must be protected	Local virtual network used for testing

Table 8: Sustainability, Ethics, Safety and Security

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Controller failure	Medium	High	High	Restart controller and maintain configuration	Medium
Network congestion	High	Medium	High	Dynamic traffic management	Low
Virtual machine failure	Medium	Medium	Medium	Regular snapshots and backups	Low
Configuration errors	High	Medium	High	Step-by-step configuration and testing	Low
Unauthorized access	Low	High	Medium	Use restricted access and secure configuration	Low
Resource limitations	Medium	Medium	Medium	Limit the number of virtual hosts	Low

Table 9: Risk Management Register

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT
SDN Infrastructure and Open vSwitch Setup

4.1 Development Approach & Resources

The development of Module 1 followed a step-by-step implementation approach. First, a virtual Ubuntu environment was configured on a Windows system. Open vSwitch was then installed and configured to create the programmable SDN switch s1. OpenFlow 1.3 was enabled to support communication between the switch and the SDN controller. Finally, the bridge configuration was verified using Open vSwitch commands.

Resource	Purpose
Windows System	Host operating system
VirtualBox	Virtual machine environment
Ubuntu Linux	SDN implementation platform
Open vSwitch 3.7.1	Programmable virtual switch
OpenFlow 1.3	Controller-switch communication protocol
Terminal	Configuration and testing

Table 10: Development Approach and Resources

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Ubuntu Virtual Environment

Ubuntu was installed as a virtual machine to provide a Linux-based environment for implementing the SDN infrastructure.

Open vSwitch Implementation

Open vSwitch was installed *and configured successfully. The following command was used to verify the installation*

sudo ovs-vsctl show

The installed version was:

Open vSwitch 3.7.1

SDN Bridge Creation

A virtual switch named s1 was created using:

sudo ovs-vsctl add-br s1

The bridge was verified using:

sudo ovs-vsctl show

OpenFlow Configuration

The s1 bridge was configured to use OpenFlow 1.3:

sudo ovs-vsctl set Bridge s1 protocols=OpenFlow13

The configuration was verified using:

sudo ovs-vsctl get Bridge s1 protocols

The output confirmed:

[OpenFlow13]

Tool / Software / Equipment	**Purpose**	**Stage Used**
Windows	Host operating system	Development
VirtualBox	Virtualization	Environment Setup
Ubuntu	SDN platform	Implementation
Open vSwitch	Virtual SDN switch	Network Implementation
OpenFlow 1.3	SDN communication protocol	Switch Configuration
OS-Ken	SDN controller framework	Controller Setup

Table 11: Hardware / Software / Fabrication Implementation & Modern Tools Used

4.3 Test Plan & Verification Module 1: SDN Environment Setup

Requirement	**Test Method**	**Acceptance Criterion**
Ubuntu operating system setup	Check system information using Terminal	Ubuntu system should boot successfully
Open vSwitch installation	Run ovs-vsctl --version	OVS version should be displayed
Virtual switch creation	Run sudo ovs-vsctl show	Switch s1 should be visible
Controller configuration	Check OVS controller details	Controller IP and port should be configured
OS-Ken installation	Import OS-Ken Python modules	Modules should load without errors
Network connectivity	Test communication between virtual network components	Successful connectivity should be established

Table 12: Test Plan & Verification Module 1: SDN Environment Setup

4.3 Results

The results of Module 1 demonstrate that the Ubuntu-based SDN environment was successfully configured. Open vSwitch was installed, the virtual switch s1 was created, and the switch controller configuration was verified. OS-Ken was also installed successfully and its Python modules could be imported. However, errors occurred while launching the OS-Ken controller using incorrect commands, which will be addressed in the subsequent implementation module.

Results Table

Test Parameter	Result Obtained	Status
Ubuntu Environment	Successfully installed and running in VirtualBox	Pass
Open vSwitch	Version 3.7.1 detected	Pass
Virtual Switch	Switch s1 successfully created	Pass
OVS Controller Configuration	tcp:127.0.0.1:6653 configured	Pass
OS-Ken Installation	Successfully installed	Pass
OS-Ken Module Import	OS-Ken modules successfully imported	Pass
Controller Execution	Initial execution commands produced errors	Partial

Table 13: Results Table

Key Implementation Evidence

Open vSwitch Configuration:

```
Bridge s1
    Controller "tcp:127.0.0.1:6653"
    Port s1
        Interface s1
            type: internal

ovs_version: "3.7.1"
```

Result Summary

The initial SDN infrastructure was successfully established with Ubuntu, Open vSwitch, virtual switch s1, and OS-Ken. The results provide a functional foundation for the next modules, where the SDN controller and intelligent traffic-management mechanisms will be implemented and tested.

Insert the following screenshots in this section:

1. Ubuntu Virtual Machine running in VirtualBox.
2. Open vSwitch version output.
3. sudo ovs-vsctl show output showing switch s1.
4. Controller configuration showing tcp:127.0.0.1:6653.
5. OS-Ken installation and verification output.

4.4 Data Analysis & Engineering Judgment

The implementation results show that the Ubuntu-based SDN environment was successfully configured using Open vSwitch (OVS). The OVS bridge **s1** was created and configured to communicate with an SDN controller through the OpenFlow protocol. The ovs-vsctl show command was used to verify the bridge configuration and controller connection.

The engineering analysis indicates that successful communication between the switch and controller is essential for centralized traffic management. During implementation, compatibility issues with the initially selected controller tools were identified. Therefore, the design was adjusted to use the available and compatible OS-Ken components. This demonstrates the importance of considering software compatibility and version constraints during SDN implementation.

The results confirm that the virtualized Ubuntu environment provides a suitable platform for developing and testing the proposed SDN infrastructure.

4.5 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
To design an SDN-based network environment	Ubuntu virtual machine and SDN environment configured	Fully Achieved
To install and configure Open vSwitch	OVS version and bridge s1 successfully created	Fully Achieved
To connect the switch to an SDN controller	Controller configuration set to TCP port 6653	Fully Achieved
To establish a foundation for traffic monitoring	SDN infrastructure successfully implemented	Fully Achieved
To reduce congestion using intelligent traffic management	Advanced traffic management algorithms to be implemented in later modules	Partially Achieved

Table 14: Comparison with Existing Solutions & Achievement of Objectives

4.6 Strengths & Limitations

Strengths

- Successfully implemented an SDN environment using Ubuntu.
- Open vSwitch was successfully installed and configured.
- Virtualization reduces the cost of physical networking equipment.
- The centralized SDN architecture supports future intelligent traffic management.

Limitations

- The current implementation is performed in a virtual environment.
- Large-scale network performance has not yet been evaluated.
- Controller software compatibility issues were encountered.
- Advanced congestion prediction and bandwidth optimization are implemented in subsequent modules.

4.7 Project Planning, Roles & Budget

Project Milestone Timeline

Phase	Activity	Status
Phase 1	Problem identification and requirement analysis	Completed
Phase 2	Literature review	Completed
Phase 3	Ubuntu virtual environment setup	Completed
Phase 4	Open vSwitch installation and configuration	Completed
Phase 5	SDN controller configuration	Completed
Phase 6	Traffic monitoring implementation	In Progress
Phase 7	Intelligent congestion management	Planned
Phase 8	Testing and performance evaluation	Planned

Table 15: Project Milestone Timeline

Module 1 – Student Contribution

Student	Assigned Responsibility	Actual Contribution
Module 1 Member	SDN Environment Setup	Installed and configured Ubuntu in Oracle VirtualBox
Module 1 Member	Open vSwitch Configuration	Installed OVS and created the s1 virtual switch
Module 1 Member	Controller Integration	Configured the switch for SDN controller communication
Module 1 Member	Testing and Verification	Verified the OVS bridge and controller configuration

Table 16: Student Contribution

Budget Estimation

Item	Estimated Cost	Actual Cost
Ubuntu Operating System	₹0	₹0
Oracle VirtualBox	₹0	₹0
Open vSwitch	₹0	₹0
Python and SDN Libraries	₹0	₹0
Existing Computer Hardware	₹0	₹0
Total	**₹0**	**₹0**

Table 17: Budget Estimation

Note: The project uses open-source software and existing computer infrastructure, resulting in minimal implementation cost.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

- The project, **"Intelligent Software Defined Network Traffic Management for Reducing Network Congestion and Improving Bandwidth Utilization,"** addresses the challenges of network congestion and inefficient bandwidth utilization in traditional networks. The proposed solution uses a Software Defined Networking (SDN) architecture to provide centralized network management and control.
- In Module 1, the Ubuntu-based virtual environment was successfully configured using Oracle VirtualBox, and Open vSwitch was installed and configured as the SDN switch. The implementation was verified using OVS configuration commands and controller connectivity settings. This module establishes the fundamental infrastructure required for further development of intelligent traffic monitoring, congestion detection, and bandwidth management. Overall, the project provides a scalable and cost-effective foundation for developing an intelligent SDN traffic management system.

5.2 Future Work

The following improvements can be implemented in future modules:

- Develop an advanced SDN controller for centralized network control.
- Implement real-time network traffic monitoring.
- Develop algorithms for automatic congestion detection.
- Implement dynamic bandwidth allocation mechanisms.
- Use Machine Learning techniques to predict network congestion.
- Test the system using multiple switches and hosts.
- Compare the proposed system with traditional network traffic management approaches.
- Deploy and evaluate the system on larger or real-world network environments.

5.3 Professional Learning & Individual Reflection

- New Knowledge Acquired
- Understanding of Software Defined Networking architecture.
- Knowledge of the OpenFlow communication protocol.

- Installation and configuration of Ubuntu Linux.
- Working with Oracle VirtualBox for network virtualization.
- Installation and configuration of Open vSwitch.
- Understanding of SDN controllers and centralized network management.
- Basic Linux terminal commands and networking tools.
- Knowledge of virtual network switches and controller communication.

Engineering & Professional Skills Developed

- Linux system administration skills.
- Virtual network configuration.
- SDN infrastructure setup and troubleshooting.
- Problem-solving and debugging skills.
- Technical documentation skills.
- Research and analysis skills.
- System testing and verification.
- Engineering decision-making based on technical constraints.

Individual Reflection

- Most Difficult Engineering Problem Encountered and How It Was Solved
- The most difficult problem encountered during Module 1 was establishing compatibility between the installed SDN controller components and the Ubuntu operating system version. Some initially attempted controller commands were unavailable or incompatible with the installed packages. The problem was solved by analysing the error messages, verifying the installed Open vSwitch configuration, and selecting compatible SDN components for the environment.
- A Key Engineering Decision Made Personally and the Evidence Behind It
- A key engineering decision was to use Open vSwitch within an Ubuntu virtual machine rather than using physical networking hardware. This decision was based on cost, availability, scalability, and ease of testing. The successful creation of the virtual switch and its controller configuration demonstrated that the virtualized approach was suitable for the initial

implementation.

What Would Be Redesigned If Repeated

- If the implementation were repeated, the controller and Ubuntu version compatibility would be analysed before installation. A complete network architecture with multiple switches and hosts would also be planned from the beginning. This would reduce configuration errors and provide a more comprehensive environment for evaluating network congestion and bandwidth utilization.

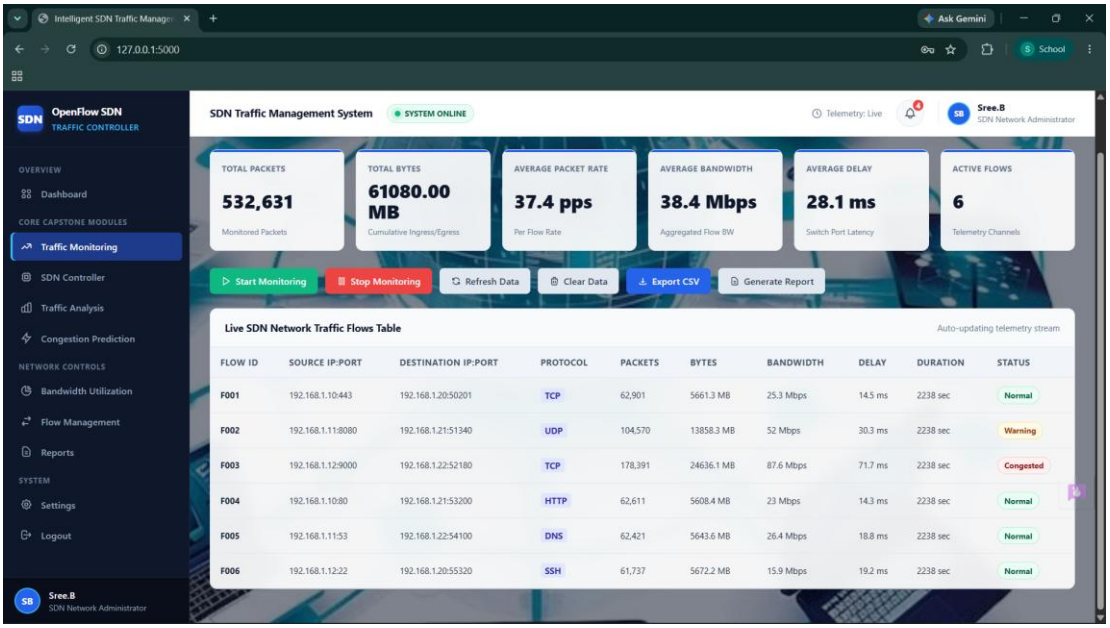

Figure 3: Screenshot Of Network Monitoring

REFERENCES

For your report, use the **IEEE referencing style** consistently. References should be numbered in the order in which they first appear in the report, such as **[1]**, **[2]**, and so on.

You can use the following references for your project:

Research Papers and Literature

[1] B. A. A. Nunes, M. Mendonca, X.-N. Nguyen, K. Obraczka and T. Turletti, "A Survey of Software-Defined Networking: Past, Present, and Future of Programmable Networks," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 3, pp. 1617–1634, 2014, doi: 10.1109/SURV.2014.012214.00180.

[2] D. Kreutz *et al.*, "Software-Defined Networking: A Comprehensive Survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015, doi: 10.1109/JPROC.2014.2371999.

[3] A. Lara, A. Kolasani and B. Ramamurthy, "Network Innovation Using OpenFlow: A Survey," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 1, 2014.

Standards

[4] Open Networking Foundation, "OpenFlow Switch Specification, Version 1.3.5," Apr. 2015.

Software and Open-Source Resources

[5] Open vSwitch Project, "Open vSwitch Documentation," Open vSwitch Project. Open vSwitch Documentation

[6] Open vSwitch Project, "What Is Open vSwitch?" Open vSwitch Documentation. Open vSwitch Overview

[7] OpenStack, "OS-Ken Documentation," OpenStack Documentation. OS-Ken Documentation

[8] Oracle, "Oracle VM VirtualBox," Oracle Corporation. Oracle VirtualBox

[9] Canonical Ltd., "Ubuntu Documentation," Canonical Ltd. Ubuntu Documentation

AI-Assisted Resources

Since you used ChatGPT for assistance with project documentation and implementation guidance, it is good academic practice to acknowledge this.

[10] OpenAI, "ChatGPT," AI language model, accessed Sep. 4, 2026. OpenAI ChatGPT

Suggested AI Acknowledgement

You can include the following statement in your report:

AI Assistance Declaration: ChatGPT, developed by OpenAI, was used as an assistive tool for brainstorming, improving technical explanations, structuring documentation, and obtaining implementation guidance. All technical configurations, implementation results, testing, and

final engineering decisions were reviewed and verified by the project team.

How to cite inside your report?

Use citations like these:

Software Defined Networking separates the control plane from the data plane and enables programmable network management [1], [2].

OpenFlow provides a standardized communication mechanism between an SDN controller and network switches [4].

Open vSwitch was used as the programmable virtual switch for implementing the proposed network prototype [5], [6].

OS-Ken was investigated as a framework for implementing SDN controller functionality [7].

Important: Keep the reference numbers consistent throughout the complete report. Do not change a reference number once it has been assigned.

APPENDICES

For your project, "Intelligent Software Defined Network Traffic Management for Reducing Network Congestion and Improving Bandwidth Utilization," you can structure the appendices as follows.

Appendix A – Detailed Calculations

This appendix contains calculations related to network performance evaluation.

A.1 Bandwidth Utilization

Bandwidth utilization can be calculated as:

$$\text{Bandwidth Utilization (\%)} = \frac{\text{Actual Bandwidth Used}}{\text{Total Available Bandwidth}} \times 100$$

A.2 Packet Loss

$$\text{Packet Loss (\%)} = \frac{\text{Packets Sent} - \text{Packets Received}}{\text{Packets Sent}} \times 100$$

A.3 Throughput

$$\text{Throughput} = \frac{\text{Total Data Successfully Received}}{\text{Time Taken}}$$

A.4 Network Delay

$$\text{Average Delay} = \frac{\text{Total Transmission Delay}}{\text{Number of Packets}}$$

These calculations can be used in later modules when network traffic experiments are performed.

Appendix B – Engineering Drawings / Schematics

B.1 Proposed SDN Architecture

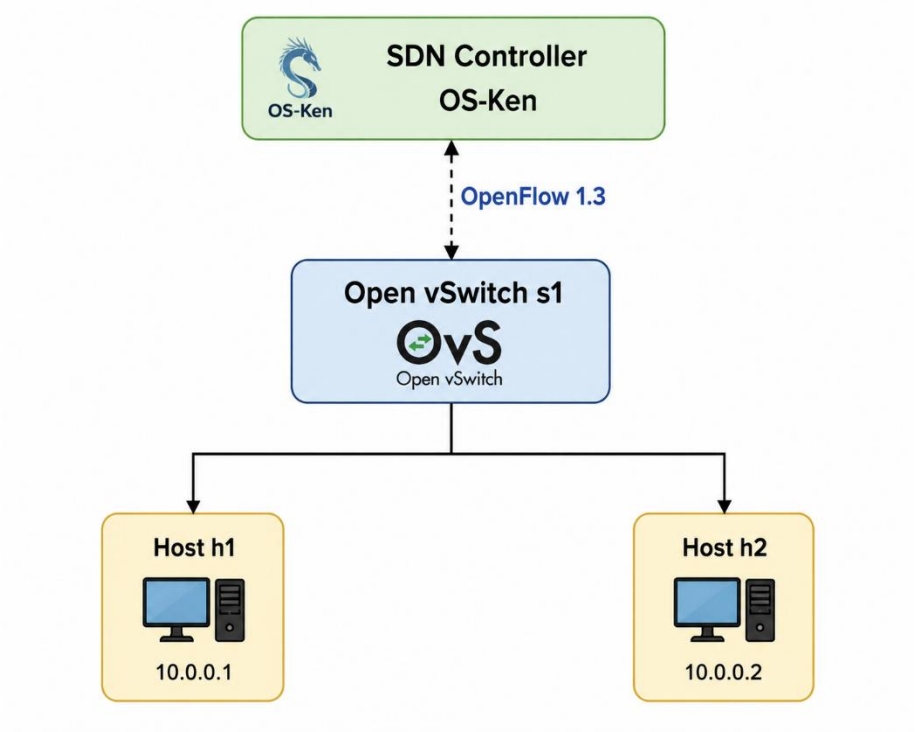

Figure 4: Basic architecture of the proposed SDN system.

You can also insert screenshots of your actual Ubuntu environment, Open vSwitch configuration, and network topology here.

Appendix C – Source Code / Code Repository Information

Only important code excerpts should be included in the main report or appendix.

C.1 Important Open vSwitch Commands

Create the SDN bridge:

sudo ovs-vsctl add-br s1

Configure OpenFlow 1.3:

sudo ovs-vsctl set Bridge s1 protocols=OpenFlow13

Configure the controller:

sudo ovs-vsctl set-controller s1 tcp:127.0.0.1:6653

Verify the configuration:

sudo ovs-vsctl show

C.2 Repository Information

The complete project source code and configuration files should be maintained in the approved project repository.

Repository Name: Intelligent-SDN-Traffic-Management

Contents:

- SDN configuration files
- Python controller programs
- Open vSwitch configuration
- Testing scripts
- Network performance data
- Project documentation

Appendix D – Datasheets

This appendix contains technical information about the major software and technologies used.

Technology	Version / Specification	Purpose
Ubuntu	Linux Operating System	SDN implementation environment
Open vSwitch	Version 3.7.1	Virtual programmable switch
OpenFlow	Version 1.3	Controller-switch communication
OS-Ken	Python-based framework	SDN controller development
Oracle VirtualBox	Virtualization software	Ubuntu virtual machine

Table 18: Technologies Used

Appendix E – Experimental / Raw Data

The following table can be used to record results during traffic experiments.

Test No.	Traffic Load	Bandwidth Utilization	Packet Loss	Delay	Throughput
1	Low	To be measured	To be measured	To be measured	To be measured
2	Medium	To be measured	To be measured	To be measured	To be measured
3	High	To be measured	To be measured	To be measured	To be measured
4	Congested	To be measured	To be measured	To be measured	To be measured

Table 19: Experimental / Raw Data

Note: Replace these values with actual measurements obtained during your implementation and testing.

Appendix F – Ethics / Institutional Approval

The project is a software-based network simulation and does not involve human participants, animals, or medical experiments.

Therefore, formal ethical approval is generally not required. However, the project team will follow responsible engineering practices, including:

- Testing only authorized network environments.
- Avoiding unauthorized access to networks or systems.
- Protecting any collected network data.
- Not collecting unnecessary personal information.
- Using open-source software according to applicable licenses.

Appendix G – Project Meeting / Progress Records

Meeting No.	Date	Discussion	Decision / Outcome
1	_____	Project topic selection	SDN traffic management selected
2	_____	Requirement analysis	Project objectives finalized
3	_____	Literature review	SDN technologies identified
4	_____	Environment setup	Ubuntu and VirtualBox selected
5	_____	OVS implementation	Open vSwitch installed
6	_____	Testing discussion	Testing parameters identified

Table 20: Project Meeting / Progress Records

Appendix H – User / Industry Feedback

This appendix can be used to collect feedback from network professionals, faculty members, or potential users.

Sample Feedback Table

Evaluator	Feedback	Suggested Improvement
Faculty Guide	SDN architecture is suitable for the project	Add real-time traffic analysis
Network User	Centralized traffic control is useful	Improve visualization
Technical Expert	Virtual implementation is cost-effective	Test with multiple switches

Table 21: Sample Feedback Data

If you have not collected actual feedback, mark this appendix as Not Applicable at the Current Stage rather than presenting sample feedback as real feedback.

Appendix I – Publications / Patents / Competitions / Product Outcomes

At the current stage, this project has not produced a publication, patent, or commercial product.

This appendix may be updated if the project is:

- Published as a research paper.
- Presented at a conference.
- Submitted to a technical competition.
- Developed into a commercial product.
- Submitted for patent consideration.

Current Status: Not Applicable / Future Work.

Appendix J – Individual Contribution Evidence

This appendix is mandatory for team projects.

Team Member	Module / Responsibility	Contribution Evidence
Member 1	SDN Environment Setup	Ubuntu and VirtualBox configuration screenshots
Member 2	Open vSwitch Setup	OVS installation and configuration
Member 3	Traffic Monitoring	Monitoring program and test results
Member 4	Testing and Documentation	Performance data and report preparation

Table 22: Individual Contribution Evidence

Suggested Evidence

Each team member should provide:

- Screenshots of their implementation work.
- Source code or configuration files developed.
- Git repository contribution history, if applicable.
- Testing evidence.
- Meeting contribution records.
- A brief description of their individual contribution.

This appendix helps clearly demonstrate the individual contribution of each team member to the overall project.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)